

Class 6: R Functions

Reta Ado (PID: A17814740)

Table of contents

Background	1
A first function	1
A <code>generate_data()</code> function	3
Write a <code>'generate_protein()</code> function	6
Are Our peptides “unique in nature?”	7

Background

All functions in R have at least 3 things:

- a *name* (we pick that and use it to call the function)
- input *arguments* (one or more comma separated inputs that go inside the brackets when we call the function),
- the “body” (the lines of R code that do the work of the function).

Q1a: Your first version of the function should add two input numbers together. For example, `add(4, 7)` should return 11.

A first function

Here we will create a function to add some numbers. Let's call it `add()`

Input arguments can be either **required** or **optional**. The later have fall-back default that will be used if the user does not specify them.

```
add <- function(x, y = 0) {  
  x + y  
}
```

Can we use our new function:

```
add(10, 100)
```

```
[1] 110
```

```
add(10)
```

```
[1] 10
```

Q1b: For your second version, adapt your first function so it can take a single input vector or two inputs as before. For example, `add(4, 7)` and `add(c(4,7),10)`.

```
add <- function(x, y = 0) {  
  sum(x, y)  
}
```

```
add(4, 7)
```

```
[1] 11
```

```
add(c(4,7, 10), )
```

```
[1] 21
```

Q1c: Finally, on your own (outside of class) create a third version of your function that can add any number of inputs that the user provides. For example, `add(1, 2, 3, -4)` should return 2. Hint: explore the `...` (dots) argument or the base R function `sum()`.

```
add <- function(...) {  
  sum(...)  
}
```

```
add(1, 2, 3, -4)
```

```
[1] 2
```

We can explicitly set a **return** value output from a function (rather than the default last line) by using the `return()` function call.

```
add<- function(x, y=0, z=0) {
  sum(x, y, z)
  cat("Is it break time yet\n")
}
```

```
add(10,100)
```

Is it break time yet

A generate_data() function

A useful function here is the “base R” `sample()` function:

```
sample(1:5, size=3)
```

```
[1] 2 4 1
```

We can use this to make a random nucleotide sequence if we draw from “A”, “C”, “G”, and “T”...

```
sample(1:5, size=60, replace = TRUE)
```

```
[1] 4 4 4 5 5 4 1 4 4 2 3 3 2 1 1 4 4 5 3 3 3 4 5 4 1 3 1 5 5 5 1 1 1 2 5 2 2 4
[39] 4 3 4 4 5 2 1 3 2 5 1 3 2 2 4 1 5 5 1 4 4 2
```

```
sample(x=c("A", "C", "G", "T"), size=10, replace = TRUE)
```

```
[1] "C" "A" "C" "A" "G" "G" "C" "T" "G" "G"
```

Q2a: Write a function `generate_dna()` that returns a random DNA sequence of a length specified by the user. Your first version should return a multi-element vector of single character nucleotides. For example `generate_dna(6)` might return “A”, “T”, “T”, “G”, “A”, “C”.

```
generate_dna <- function(len) {
  sample(x=c("A", "C", "G", "T"), size=len, replace = TRUE)
}
```

```
generate_dna(len=100)
```

```
[1] "G" "T" "C" "C" "T" "C" "T" "G" "C" "T" "A" "G" "T" "A" "G" "T" "T" "T"  
[19] "T" "T" "G" "C" "G" "T" "C" "G" "T" "G" "C" "C" "T" "C" "C" "A" "C" "G"  
[37] "C" "A" "A" "T" "T" "T" "G" "G" "A" "G" "C" "G" "G" "T" "G" "G" "G" "C"  
[55] "C" "C" "G" "T" "G" "A" "C" "A" "T" "A" "G" "G" "A" "G" "A" "T" "A" "T"  
[73] "G" "T" "G" "G" "G" "T" "C" "G" "T" "A" "A" "T" "A" "A" "G" "G" "A" "C"  
[91] "G" "T" "G" "T" "C" "A" "A" "C" "A" "T"
```

```
generate_dna()
```

```
[1] "A" "A" "T" "A"
```

Q2b: Your second version should optionally be able to return either a multi-element vector of single character nucleotides (as before) or a single character string (not a vector of single letters but a single vector of multiple letters). For example “AAGGCTG”.

```
generate_dna <- function(len, single.element = FALSE) {  
  ans <- sample(x = c("A", "C", "G", "T"), size = len, replace = TRUE)  
  
  if (single.element == TRUE) {  
    ans <- paste(ans, collapse = "")  
  }  
  
  return(ans)  
}
```

```
#example of expected behavior  
generate_dna(6)
```

```
[1] "T" "T" "G" "A" "T" "G"
```

```
#example of single character string  
generate_dna(6, single.element = TRUE)
```

```
[1] "CGGTAA"
```

Functions that could be useful here are `paste()`, `if()`, `cat()`, and `return()`

```
paste(c("A", "C", "G"), collapse = "---" )
```

```
[1] "A---C---G"
```

Q2c: Finally, create a final version of your function that prints out a FASTA format sequence with an id line indicating the sequence length.

```
len9  
CGAAGGCTG
```

```
cat("hello \t there")
```

```
hello      there
```

```
generate_dna <- function(len=10, single.element=TRUE) {  
  
  ans<- sample(x=c("A", "C", "G", "T"), size=len, replace = TRUE)  
  #cat("Hello....")  
  
  if(single.element) {  
    #cat("is it me you are looking for...")  
    ans <- paste( ans, collapse = "")  
  }  
  
  ## Format as FASTA with an ID line  
  
  cat( paste(">len", len, "\n", sep=""))  
  cat(ans)  
  cat("\n")  
  ##  
  
  return(ans)  
}
```

```
x <- generate_dna(20)
```

```
>len20  
GCAATAAGTTGCTCCTTTAG
```

Write a 'generate_protein()' function

Q3: Write a function `generate_protein()` that returns a random peptide/protein sequence of a length specified by the user. For example `generate_protein(6)` might return "WQRTAG".

```
generate_protein <- function(len) {  
  
  # Write out all 20 standard amino acids using single letter code  
  aa <- c("A", "R", "N", "D", "C", "E", "Q", "G", "H", "I", "L", "K", "M", "F", "P", "S", "T")  
  
  # Randomly sample amino acids with replacement to build a protein sequence  
  # size = len means the sequence will be as long as the user specifies  
  ans <- sample(x = aa, size = len, replace = TRUE)  
  
  # Collapse the vector into a single string  
  paste(ans, collapse = "")  
  
}
```

```
# Example of the expected behavior  
generate_protein(6) # e.g. "WQRTAG"
```

```
[1] "TVFQMW"
```

Q4: Adapt and use your `generate_protein()` function to generate a series of random protein sequences ranging from 6 to 13 amino acids in length (one sequence per length). Take advantage of the base R function `for()` or `sapply()` so that you do not have to call `generate_protein()` eight times by hand.

```
for(l in 6:13) {  
  cat(">id.", l, "\n", sep = "")  
  cat(generate_protein(l), "\n")  
  
}
```

```
>id.6  
SGILTG  
>id.7  
DCARQVW
```

```

>id.8
PNNADAI
>id.9
FKISLIHLK
>id.10
WAQTGPFADR
>id.11
YWPAYNIMYTE
>id.12
DFLWVMGKCRES
>id.13
FRYLIFLYECPV

```

```
generate_protein(6)
```

```
[1] "DNTLAY"
```

Are Our peptides “unique in nature?”

Q5: Take your FASTA-formatted peptides from Q4 and run them as a single BLASTp search against the Non-redundant protein sequences (nr) database at <https://blast.ncbi.nlm.nih.gov/>. For this question do not restrict the organism (leave the Organism field blank so that the full nr database is searched).

Length	Ide	Cov	Uniq
6	100	100	N
7	100	100	N
8	100	100	N
9	100	89	Y
10	88.89	90	Y
11	81.82	100	Y
12	100	75	Y
13	81.82	85	Y

Q5a: At which sequence length do your randomly generated peptides start to look “unique in nature” (i.e. no 100% coverage + 100% identity hit)?

They start to look unique in nature after the 8th sequence length (starts at 9).

Q5b: Speculate why very short random peptides are almost always found in nr, while longer ones typically are not. Your answer should refer both to the size of the sequence space (20^L for a peptide of length L) and to the size of the known protein universe.

Short random peptides are almost always found in nr because their sequence space (20^L) is very small compared to n (number of sequences in known protein universe), which makes it likely that any randomly generated sequence has been sampled by a natural protein. As peptide length (L) increases, the sequence space grows exponentially while the number of known proteins n remains fixed. Once sequence space exceeds n , the vast majority of possible sequences haven't appeared in any natural protein, making it increasingly unlikely that a randomly generated peptide of length L would be found in the database.

Q6: In 3–6 sentences total and using your Q5 data and the reasoning from Q5b, what do you think this minimum length is and why might it be a bad design choice for the immune system to present very short peptides?

The minimum useful peptide length is around 9 amino acids since this is where sequences first become unique in nature. Presenting very short peptides (like 6,7,8 in my example) would be a bad design choice for the immune system because their sequence space is too small, meaning those sequences are too short to be distinctive from naturally occurring proteins. Therefore, the body's immune system does not have ability to determine pathogens from self peptides, triggering possibly bad autoimmune responses. Longer peptides have a larger sequence space (ex: 20^9), making it much more likely that a peptide is unique to a pathogen rather than shared with a host protein.